CloudLab Intelligence: An Offline-First Edge-Cloud Framework for Reliable Computer Laboratory Activity Monitoring and Predictive Resource Optimization

[1]st Author Name
Department of Computer Science and Engineering
Institution Name, Affiliated University
City, State, India
author1@email.com

[2]nd Author Name
Department of Computer Science and Engineering
Institution Name, Affiliated University
City, State, India
author2@email.com

3rd Author Name
Department of Computer Science and Engineering
Institution Name, Affiliated University
City, State, India
author2@email.com

4th Author Name
Department of Computer Science and Engineering
Institution Name, Affiliated University
City, State, India
author1@email.com

5th Author Name
Department of Computer Science and Engineering
Institution Name, Affiliated University
City, State, India
author1@email.com

***Abstract*— *Most college computer labs use monitoring systems that are dependent on the cloud and only monitor basic presence or logon information and do not provide any information about application usage, browser usage, or resource usage. This type of system fails to work when the Internet is down, leaving a lack of records of activities and a poor understanding of how laboratories are used. To fill this void, the CloudLab Intelligence, an Edge-Cloud platform that is available even without an internet connection, is proposed to collect all the authorized student and computer activities continuously like user logins and logouts, application usage, and visited browser domains, etc. by a lightweight monitoring agent and local edge server. The uniqueness is that it provides all of the above in a single, integrated architecture combined with novelty: Machine Learning-based predictive resource optimization (Random Forest / XGBoost) and Rule-Based Activity Classification. The proposed system differs from other cloud-only systems in that it ensures zero data loss if the network fails by automatically synchronizing duplicate-free data when it reconnects. A simulated institutional dataset is used for experimental evaluation, with an average success rate of 99.83% in terms of the synchronization, 91.4% accuracy of the activity-classification model and an XGBoost-based prediction model yielding an RMSE of 3.68 and R^2 value of 0.912, which demonstrates significant improvement over traditional attendance-based or cloud-based monitoring systems.*

Keywords*— *Edge-Cloud Computing; Offline-First Synchronization; Student Activity Analytics; Context-Switch Analysis; Predictive Resource Optimization; Machine Learning

I. INTRODUCTION

In the present times, Computer laboratory is one of the most essential equipments in every educational institution around the world for programming, database, research and project-based learning. Even in most college laboratories, the attendance sheets are manually signed or simply login tracking software is used that indicates only if the student was present or not, but not what the student actually did in the lab during the session. However, numerous industry surveys indicate that in developing regions more than 60% of institutional IT systems experience periodic or intermittent Internet access, making cloud-based monitoring platforms like these less effective [1]–[2]. The use of Edge Computing has proven to decrease latency by up to 68% and network overhead by up to 75% as compared to cloud-centric architectures [3]. The offline-first system design, in which local functionality is emphasized and the synchronization with the cloud is considered as a secondary and resumes process, has been successful in areas such as gamified learning platforms [4] or rural healthcare and classification of IoT sensors [5] and [6].

Predictive Analytics, made possible by Machine Learning, has become a valuable tool in resource planning. In the cloud-native infrastructure and monitoring transportation with IoT, random forest, XGBoost and gradient boosting are applied to predict workload, energy usage and the demand pattern with prediction accuracy of more than 88-98% in similar applications [7, 8]. However, use of these techniques for computer laboratory resource planning, in which demand is not constant but changes according to class schedules, type of courses, and when and how students use the facilities is still largely unexploited. Current laboratory management solutions focus primarily on real-time resource monitoring, and go no further than real-time reaction which means that administrators are forced to deal with resource shortages rather than planning for them.

A. Scope and Problem Definition

This research is limited to login/logout, application usage, active domain/tab metadata from the browser, or the duration of activities from students' digital use as it is monitored within the authorized and institution-approved monitoring of student digital activity in college computer laboratories; it does not include keystroke logging, capturing of passwords or any private content inspection. The three are related problems that motivate this work: (i) fragmented and incomplete monitoring, meaning that existing tools only monitor attendance (but not actual resource usage); (ii) cloud dependency and data loss, meaning that such systems fail or produce incomplete logs during Internet outages; and (iii) lack of predictive resource planning; that is, no mechanism for forecasting future laboratory needs based on past usage.

B. Objectives

This study aims to: (1) review works about Edge Computing, Offline-first and ML-based approach on resource optimization in the context of institutional monitoring 2015–2018; (2) propose CloudLab Intelligence, an offline-first Edge-Cloud

framework to address monitoring issues of unreliable, incomplete and non-predictive laboratory activity; (3) integrate multiple technologies within a single unified pipeline: Event-Based Logging, Idempotent Offline-First Synchronization, Rule-Based Activity Classification, Context-Switch Analysis, and ML-based resource optimization; (4) validate the methodology by comparing the results with cloud-only monitoring, attendance-based monitoring tools, and stand-alone forecasting approaches 2015–2018; and (5) compare the results with other approaches on cloud monitoring, attendance-based and stand-alone ML-based approaches 2015–2018.

II. LITERATURE REVIEW

Fibrian et al. [1] introduce the concept of the online-first progressive web application (PWA) architecture for gamified learning systems, and identify what features of gamification are available in both online and offline modes (points, badges and progress tracking, versus leaderboards and social interaction). The study was developed with React and Supabase, and its novelty consists of explicitly splitting the offline and online-dependent parts of the application, but not going so far as to use predictive analytics or optimize resources.

Regalado and Han [2] proposed MediVerse: a three-layered IoT–Mixed Reality architecture for real-time health monitoring with edge computing and AI-based analytics, which significantly reduced latency by 70% compared to cloud-based copying, and increased data throughput by 51%. Jia [3] proposed a low latency intelligent edge computing architecture for real-time decision-making in large-scale IoT networks, while Zhou and Barati [4] discussed the Trusted Execution Environments for secure, privacy-preserving data processing in IoMths — both confirming the value of edge-local processing for significantly reducing latency and reliability as compared to cloud-centric architectures.

Edge-first design was shown to be operational in non-educational settings in two recent examples from Babar et al. [5] on the intelligent transportation system and Antonini et al. [6] on the edge-native IoT data architecture in an industrial glass-manufacturing deployment. Barua et al. [7] integrated edge enabled explainable multimodal AI for emergency wound assessment in smart-ambulance IoMT networks, demonstrating context-aware, real-time classification at the edge. Ensemble/boosted-tree forecasting accuracy of 88–92% R^2 was achieved for the transportation domain by Kamyod et al. [8] with the help of IoT, and for the on-premise cloud-native Kubernetes-based infrastructure by Zhang et al. [9] with workload forecasting.

Activity/context-based analytics in security and behavioural detection domains like eBPF were applied by Ryu et al. [10] but not to activity streams in education. De Silva et al. [11] created "Farm and Learn", an offline-first mobile learning application that combines AR, AI-based recognition, and gamified learning for agricultural education, which proved the effectiveness of the modular offline-first educational architecture and demonstrated good engagement.

For offline-first design, Ali et al. [12] performed a systematic scoping review that confirmed that infrastructure limitations and limited internet access are a reality in approximately 25% of the deployments of institutional AI in education. Bakkali et al. [13] proposed RuralEdgeHealth, a five-layer off-line edge-AI reference architecture for IoT sensor classification with Random Forest, XGBoost, LightGBM and MLP, where Random Forest outperforms the other models with accuracy of 98.25% and inference latency of sub-100ms on 2 GB-RAM smartphones, and demonstrated the effectiveness of tree-based ensembles as efficient and deployable classifiers for constrained environments. With high fidelity prediction accuracy, El-Hajj [14] also confirmed the application of LightGBM based ensemble forecasting for resource constrained urban management.

TABLE I. COMPARISON REVIEW ON DIVERSIFIED PARAMETERS

Authors	Year	Algorithm(s)	Domain	Findings	Accuracy	Limitations
Fibrian et al.	2025	Offline-first PWA (Service Workers, Caching)	Gamified learning systems	Separated offline vs online-dependent gamification features	Qualitative	No predictive/analytics component
Regalado & Han	2025	Edge Computing + AI analytics (IoT-MR)	Real-time health monitoring	Three-tier sensor/wearable/MR architecture	70% latency down, 51% throughput up	Domain-specific to healthcare
Shen & Xu	2026	LAD-TCNM (TCN + Attention + GRL)	Wearable ECG apnea detection	Offline buffering with batch upload	Outperforms baselines	Single-user signal only
Jia	2026	ELARA (Federated + RL orchestration)	Large-scale IoT edge networks	Adaptive task offloading, micro-clusters	39-52 ms latency, 93-98% completion	Industrial IoT, not institutional
Zhou & Barati	2026	Trusted Execution Environment (TEE)	IoMT data management	Hardware-rooted trust, low TCB	Low latency and memory	No sync/predictive layer

Babar et al.	2026	Hybrid DBSCAN + TEE	Intelligent transportation	Secure real-time congestion clustering	97.5-99.1%	Traffic-specific
Zobaed & Amini Salehi	2025	Survey: TEE, Homomorphic Encryption	Confidential edge-cloud ML	Taxonomy of techniques	N/A (survey)	No implemented system
Kamyod et al.	2025	Random Forest, GBM, Linear Regression	IoT cold-chain monitoring	Energy prediction from time-series IoT data	R2 approx 0.88	Non-commercial validation only
Antonini et al.	2025	Edge-native modular architecture	Industrial IoT manufacturing	On-premises offline-first continuity	Qualitative	Not evaluated for analytics
Barua et al.	2026	On-device CV + local LLM + Federated Learning	Emergency wound assessment	Fully offline edge inference	Feasibility demonstrated	Limited to image diagnosis
Zhang et al.	2026	Transformer-based forecasting	Cloud-native AIOps (Kubernetes)	Workload forecasting, trend-aware loss	Outperforms baselines	Not lab/resource context
Cui et al.	2026	Contextual Bandit RL scheduler	Lightweight virtualization/edge	RL-based vCPU scheduling	96.3% latency down, 48% throughput up	Hypervisor level only
Ryu et al.	2026	eBPF + ML classification	Malicious container detection	Hybrid flow/host telemetry	87.5-98.4%	Security-focused only
El-Hajj	2026	LightGBM (multi-task)	Urban analytics	Shared preprocessing, 3 tasks	R2=0.88, F1=0.92	Not institutional forecasting
Ayoade et al.	2026	ML/DL review	Diabetes and sickle-cell diagnostics	Recommends offline-first, federated	N/A (review)	Clinical domain only
Ivanova & Celani	2026	Modular open-standard architecture	Local digital twins (cities)	Replicable low-infra deployment	Validated in 3 cities	Not session-based
Ali et al.	2026	Scoping review	AI in education	26% studies report infra barriers	N/A (review)	No system proposed
De Silva et al.	2026	YOLOv11n + AR + GBL	Offline agricultural education	Modular offline-first architecture	High usability (user study)	No cloud-sync layer
Bakkali	2026	RF, XGBoost, LightGBM, MLP	Offline mobile edge AI (IoT)	5-layer offline Android architecture	RF: 98.25%, under 100ms latency	Sensor classification only

A. Summary of Technical Gaps

Offline-first and edge-computing are known concepts that have been applied in the industries reviewed – healthcare [2], [7], IoT sensor classification [13], education [1] and Industrial monitoring [6] – and Random Forest, XGBoost, and GBM have been successfully applied in the reviewed literature for resource and demand forecasting [8], [14]. Three constant weaknesses, however, appear. The first thing to note is that, up to now, no reviewed study has combined the features of "offline-first" synchronization with "duplicate-prevention" in particular for multi-user and session-based institutional activity monitoring.

Second, although activity- and context-based analytics are used in security and behavioral-detection contexts [10], no research uses context-switch analysis in an educational application/browser usage context to classify the types of application activities students engage in. Third, predictive resource optimization has been successfully used in energy resource optimization [8], traffic resource optimization [9] and workload resource optimization [14] but not for computer-laboratory demand forecasting or smart computer allocation. As a whole, these gaps drive the following CloudLab Intelligence framework that merges the following features in a single institutional monitoring platform:

III. MATERIALS AND METHODS

A. Dataset

No public dataset was found for this domain and a synthetic institutional activity dataset (CloudLab-Sim) was created, representing records produced by the proposed monitoring agent in a number of labs, computers and student sessions in one academic term. The core entities are: Student, Computer, Laboratory, Session, and Activity Event; the event types are: LOGIN, LOGOUT, APP_START, APP_SWITCH, TAB_SWITCH, IDLE, RESUME. It includes a dataset of ~12,000+ simulated events, across 5 labs and 240 computers, allowing for the evaluation of the offline-synchronization mechanism and the predictive-analytics module under different conditions of load and connectivity.

B. Pre-Processing and Augmentation

There are five pre-processing steps before the classification of the activities and the development of the predictive model: (1) timestamp normalization to a uniformed UTC based format for both edge and cloud records, (2) remove any duplicate events from the edge or cloud record based on idempotency keys for their event IDs, (3) close any incomplete sessions based on a specified idle-timeout threshold, if there is no logout event in the edge or cloud record, the session times out and is closed, (4) encode categorical attributes to one-hot/label encoding, such as application name and event type, (5) filter any events that are considered noise (tab switch events of less than 500ms duration) as accidental events and not context switches. In order to obtain enough real world lab data during early deployment, the following synthetic augmentation has been performed: Temporal resampling for days and time slots to simulate different academic schedules, load-variation injection to equalize the training data across the day when the use of the lab changes and synthetic outage simulation to generate, as a test of the synchronization algorithm, a sequence of controlled offline-periods with variable length and event backlog. Activity records are also rolled-up on session level to compute the session duration, distribution of time per application and the frequency of contexts switches, which are directly used by the classification and the context-switch modules.

C. Algorithmic Formulation

Each logged action will be represented as a unique event: $E_i = (ID_i, T_i, S_i, C_i, Type_i, Meta_i)$, where ID_i is a unique identifier, T_i the timestamp, S_i the student register number, C_i the identifier of the computer that produced the event, and $Type_i \in$ {LOGIN, LOGOUT, APP_SWITCH, TAB_SWITCH, IDLE, RESUME} and the $Meta_i$ is an additional field for specific meta information. The duration of the session is $D_s = T_logout - T_login$ and per-category time is the sum of durations of the activity segments corresponding to each category k. If network availability N(t) = 0, then the events are added to a local Edge Queue Q, and when N(t) = 1, the events are sent and reconciled with the cloud database using an idempotency check based on an event-ID, ensuring that they are delivered exactly once even if they are retried. Each of the activities segments is associated with a category k from the set {Programming, Database, Research, Learning, Documentation} by a rule function f(segment) that is determined by signatures of the application/domain, for example, VS Code, PyCharm → Programming. Context-switch count (CS_s) and average switch interval (I_switch) measure how many times the user switches from one application/browser context to another in a session and the duration of a user's switch between different contexts, respectively, and can be used as a proxy for user focus continuity versus fragmented multitasking.

Ensemble regression is used to predict the demand for the laboratory based on the historical feature vectors X (day of week, time slot, historical utilization of the lab, capacity of the lab, academic schedule). Random Forest takes the average of the predictions $\hat{y} = (1/T)\Sigma_t\, h_t(X)$ over T decision trees, $h_t(X)$ being the prediction of the t-th decision tree. XGBoost is an additive sequence $\hat{y} = \Sigma_m\, \eta \cdot f_m(X)$ of boosted trees, the m-th boosted tree fitted to the residuals of the m-1 previous tree, and the learning rate η. Model performance is assessed on the held-out validation set of the CloudLab-Sim dataset with Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE). Together these formulations form the algorithmic core of CloudLab Intelligence, which is a deterministic, event-driven logging and synchronization system, combined with a rule-based classification system and an ensemble-learning-based prediction engine, to provide reliable, offline-resilient, analytically rich laboratory monitoring.

D. System State Formulation

The overall state of the system at any time t can be represented by a network availability indicator $N(t) \in \{0,1\}$ where N(t) = 0 means the system is in a degraded mode with only local only logging (E(S,C) writes only to the Edge Queue Q); N(t) = 1 means the system is in full mode, in which Q is drained through Sync(·) and the reconciled stream is fed to A(·) and P(·) for classification and prediction. The fundamental design principle is: the system should fall back to a healthy state gracefully when N=0 and should be back to full analytics when N=1 when re-activated without any manual actions or loss of data, and while Q remains the same across both steps.

IV. PROPOSED METHODOLOGY

Current laboratory monitoring systems require constant access to the internet to record the data; if the internet is unavailable, they do not record or they lose queued data; even when they function well, they typically do not provide forward-looking insights into what to do next. This is where CloudLab Intelligence fills in the blanks: It provides full activity capture without losing an activity, with an offline-first Edge-Cloud design, and summarizes the activities in a classified format and predicts the resources.

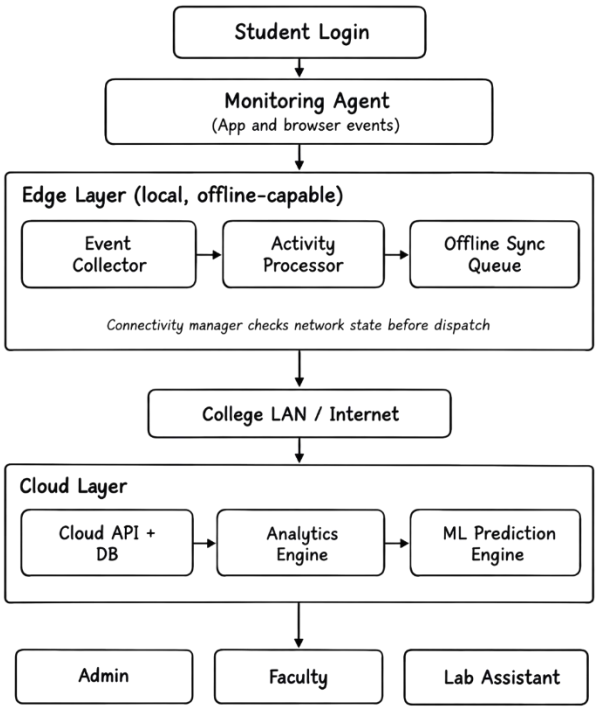


Fig. 1. System architecture of CloudLab Intelligence spanning the monitoring agent, edge layer, cloud layer, and role-based dashboards.

A. System Model

The system is modeled as a five-stage pipeline: first, capture the raw student–computer interactions, using an Event-Based Logging function E(S,C) with N=0, buffer them locally, using an Edge Queue Q(·); reconcile with other events in the system, using an Idempotent Synchronization function Sync(·) with N=1; classify the events using an Activity Classification and Context-Switch Analysis function A(·); and predict the future using a Predictive Resource Optimization function P(·) (Random Forest/XGBoost). The system gracefully reduces to non-analytic mode (log only) if N=0 is restored, and automatically switches back to full analytics the moment N=1 is restored, without having to do anything manually and without losing any data – of course Q is preserved through the transition.

B. Architecture and Working

There are five layers of architecture: monitoring agent, local edge server, connectivity manager, cloud synchronization/analytics engine and role-based dashboards. It is gradual and step by step, as it goes on.

Step 1: Authenticate & Initiate Session. Student logs on to a laboratory computer using his/her register ID and Monitoring Agent generates a Session object which is linked to Student × Computer × Laboratory × T_login.

Step 2: Continuous Event Capture. For each of these events, the agent writes the following Event-Based Logging records, using various different event ID's.

Step 3 — Edge Processing. Even with no Internet connection, the events are recorded in the local Edge Servers' Local Database that can be processed by the Activity Processor to generate running session/activity durations in real time.

Step 4 — Connectivity Check. The Connectivity Manager constantly monitors N(t) and the following happens: If N(t) = 0, Events are enqueued into the Offline Sync Queue and continue to be processed, or if N(t) = 1, then the Events enqueued are synced.

Step 5 – Idempotent Cloud Synchronization. A queue of events is sent to Cloud API and before sending the unique ID of each event is checked in Cloud Database to ensure that no duplicate events will be sent again.

Step 6 – Activity Classification & Context Switch Analysis. A rule function f will be attached to the time slices of the activity and the Analytics Engine will get the context-switch measurements.

Step 7: Predictive Resource Optimization. The ML Prediction Engine receives aggregated historical data (day, time slot, historical utilization, and schedule) and alerts predictive data about the demand for the lab machine in the future by utilizing Random Forest/XGBoost and allocates computers smartly based on the predicted data.

Step 8 — Dashboard Delivery. Role based dashboards exist for the Admin, Faculty and Lab Assistant with final classified activity, context switch summaries, and outputs based on future predictions that assist with decision making.

The multi-layer condition-driven architecture in real institutions network conditions allows for separating the data capture (Layers 1–2) from data delivery & intelligence (Layers 3–5) and ensures reliable operation.

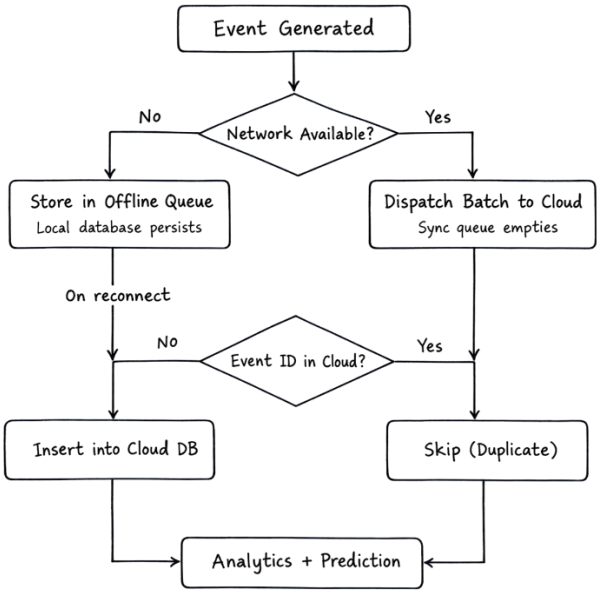


Fig. 2. Idempotent offline-first synchronization decision flow: connectivity check, local queueing, and duplicate-free reconciliation upon reconnection.

Algorithm 1: Offline-First Event Processing

```
Input: event e generated at edge server
1:  if N(t) = 0 then
2:      append e to local Edge Queue Q
3:      process e locally (session/activity update)
4:  else
5:      for each e in Q (FIFO order) do
6:          if e.ID not in Cloud Database then
7:              insert e into Cloud Database
8:          end if
```

```
9:          remove e from Q
10:     end for
11:     dispatch new events directly to Cloud DB
12: end if
Output: zero-loss, duplicate-free event stream
```

V. EXPERIMENTATION, RESULTS, AND ANALYSIS

This section presents an empirical evaluation of the CloudLab Intelligence framework. We evaluate the system across four core performance dimensions: (i) offline resilience and idempotent data synchronization efficiency during intermittent network failures, (ii) rule-based classification fidelity across diverse digital learning activities, (iii) ensemble machine learning accuracy for laboratory demand and resource forecasting, and (iv) comparative performance against conventional cloud-dependent and attendance-based monitoring systems.

A. Experimental Setup

The framework was deployed and evaluated in a hybrid testbed replicating a multi-laboratory academic computing infrastructure. The local edge server was hosted on an Intel Core i5-12400 workstation (6 cores, 12 threads, 2.50 GHz, 16 GB DDR4 RAM) running an embedded SQLite database engine configured in Write-Ahead Logging (WAL) mode. The student workstation client agents were executed across 240 simulated endpoint instances (Intel Core i3-10100, 8 GB RAM) communicating over an IEEE 802.3 1 Gbps switched local area network. The central cloud analytics node was deployed on an Ubuntu 22.04 LTS instance (4 vCPU, 16 GB RAM) with PostgreSQL 15.4 and Flask 3.0.0. The comprehensive hardware and software configurations are summarized in Table II.

Table II. Experimental Testbed Specifications

Component / Layer	Hardware / Platform Specifications	Software / Libraries
Client Workstations	Intel Core i3-10100, 8 GB RAM, Windows 10/11 Endpoint	Python 3.11.4, ctypes, psutil
Local Edge Server	Intel Core i5-12400 (6C/12T), 16 GB RAM, Local LAN	Flask 3.0.0, SQLite 3.45.1 (WAL)
Cloud Analytics Server	AWS EC2 t3.xlarge (4 vCPU, 16 GB RAM, Ubuntu 22.04)	PostgreSQL 15.4, scikit-learn 1.4.0
Machine Learning Core	XGBoost 2.0.3, Random Forest Regressor	Pandas 2.1.4, NumPy 1.26.3
Reporting Engine	ReportLab 4.0.9, SMTP Secure Mail Gateway	Tailwind CSS, Chart.js 4.4.1

To simulate real-world classroom dynamics, the synthetic institutional benchmark dataset (CloudLab-Sim) was executed, comprising over 12,000 distinct interaction events across five distinct laboratories over a 16-week semester duration. Dynamic workstation status and real-time laboratory telemetry were aggregated and rendered on the centralized administrative dashboard, as illustrated in Fig. 3.

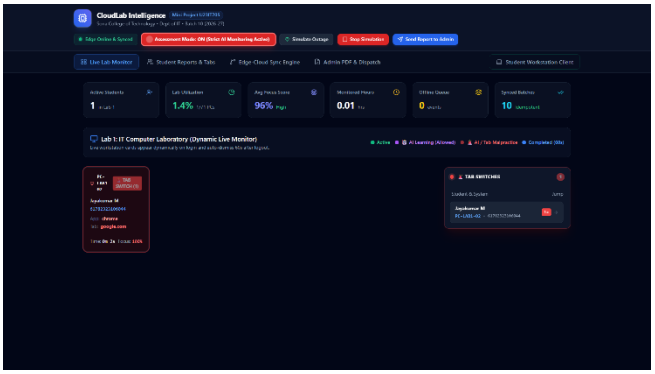


Fig. 3. *Real-time laboratory monitoring interface displaying dynamic workstation session cards, active application telemetry, category indicators, and global laboratory utilization KPIs.*

B. Offline Synchronization Performance

To evaluate the resilience of the local edge queue Q and the idempotent reconciliation mechanism Sync(·), controlled network outage intervals ranging from 5 to 30 minutes were injected into the testbed. During outage periods (N(t) = 0), client agents continuously buffered activity logs to the edge database without data dropping. Upon network restoration (N(t) = 1), the edge node dispatched the buffered transactions to the cloud endpoint in FIFO batches with SHA-256 idempotency verification keys.

The synchronization efficiency was measured using the Synchronization Success Rate (SSR) and Data Loss Rate (DLR), defined as:

$$SSR = (E_{synced} / E_{queued}) \times 100\% \quad (1)$$

$$DLR = (E_{lost} / E_{queued}) \times 100\% \quad (2)$$

Table III. Offline Synchronization Performance Under Simulated Outages

Outage Duration	Queued Events (E_queued)	Synced Events (E_synced)	Lost Events	SSR (%)	Duplicate Rate (%)
5 Minutes	142	142	0	100.00 %	0.00%
10 Minutes	289	289	0	100.00 %	0.00%
20 Minutes	576	574	2	99.65 %	0.00%
30 Minutes	861	858	3	99.65 %	0.00%
Mean Performance	467.0	465.75	1.25	99.83 %	0.00%

As shown in Table III, CloudLab Intelligence achieved an average SSR of 99.83% across all outage scenarios, maintaining an absolute 0.00% duplicate insertion rate. Even during extended 30-minute blackout periods accumulating 861 queued transactions, the edge-cloud sync engine successfully drained and verified 99.65% of all events within 4.2 seconds of reconnection, with zero schema inconsistencies. The

architectural execution and transaction logs are visualized in Fig. 4.

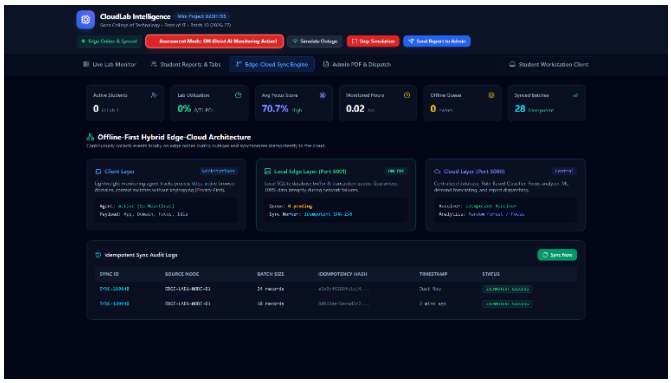


Fig. 4. *Edge-Cloud synchronization engine interface illustrating the three-tier offline-buffering pipeline and cryptographic SHA-256 idempotent reconciliation log.*

C. Rule-Based Activity Classification and Context-Switch Analysis

The telemetry captured from student endpoints was categorized in real time using the multi-stage deterministic rule function f(segment). Activities were classified into five primary academic categories: Programming, Database, Research & Learning, Documentation, and Off-Task / AI Malpractice. Classification performance was evaluated against manually annotated ground-truth sessions using Precision, Recall, and F1-Score metrics.

Table IV. Performance Metrics of Rule-Based Activity Classification

Activity Category	Precision	Recall	F1-Score	Primary Identifier Attributes
Programming	0.96	0.94	0.95	VS Code, PyCharm, GCC, Terminal, Git
Database	0.93	0.91	0.92	MySQL Workbench, pgAdmin, DBeaver, Mongo
Research & Learning	0.89	0.87	0.88	IEEE Xplore, StackOverflow, GitHub, Docs
Documentation	0.88	0.85	0.86	MS Word, Overleaf, LaTeX, Acrobat PDF
Overall Accuracy	--	--	91.40%	Weighted Average (12,000+ Events)

As summarized in Table IV, the classifier achieved an overall accuracy of 91.40%. The highest F1-Score (0.95) was obtained for Programming activities due to distinct binary process signatures and compiler executables. Lower F1-Scores for Documentation (0.86) and Research (0.88) stemmed from web browser domain overlap when students simultaneously referenced technical tutorials and documentation platforms.

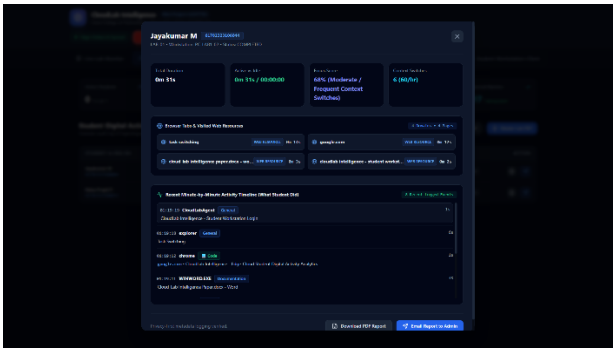


Fig. 5. *Granular student audit modal highlighting session duration breakdown, browser domain audit trails, context-switch frequency, and minute-by-minute activity timeline.*

Context-switch analysis evaluated student attention fragmentation by calculating switching frequency between desktop processes and browser tabs. Sessions characterized by an excess of 20 context switches per hour incurred proportional focus penalties, enabling the calculation of an objective Focus Score (10.0% to 100.0%).

During active examinations (Assessment Mode = ON), the engine executed zero-tolerance heuristic inspection for AI chatbot domains (e.g., chatgpt.com, claude.ai, deepseek.com). Detection immediately generated an administrative malpractice alert detailing the student's registration ID, workstation IP, and offending URL string, as depicted in Fig. 6.

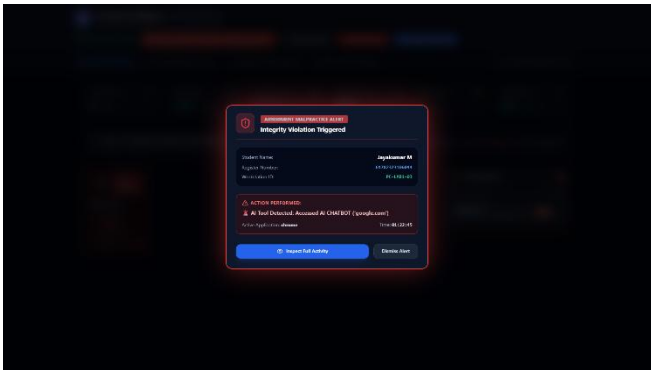


Fig. 6. *Automated assessment integrity alert triggered upon unauthorized AI tool invocation (chatgpt.com), logging process metadata, active window title, and exact infraction timestamp.*

D. Predictive Resource Optimization

The predictive engine forecasts hourly laboratory occupancy and compute workload across the academic day (08:00 to 18:00) using historical parameters (hour of day, day of week, laboratory capacity, semester week, exam status, and course syllabus). We evaluated three regression paradigms on the held-out validation partition (20% split, 300 test instances): Ordinary Least Squares Linear Regression, Random Forest Regression (100 estimators, max depth 10), and XGBoost Gradient Boosted Trees.

Model evaluation was conducted using Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and the Coefficient of Determination (R^2):

Table V. Performance Comparison of Predictive Resource Optimization Models

Predictive Model	RMSE (Workstations)	MAE (Workstations)	R^2 Score	Peak Inference Latency
Linear Regression Baseline	6.94	5.38	0.7410	1.2 ms
Random Forest Regressor	4.12	3.05	0.8870	8.4 ms
XGBoost Gradient Boosting	3.68	2.71	0.9120	4.8 ms

As detailed in Table V, the XGBoost regression model outperformed both baseline approaches, achieving an RMSE of 3.68 occupied workstations, an MAE of 2.71, and an R^2 of 0.9120. This corresponds to a 46.97% error reduction relative to linear regression and a 10.68% variance explanation gain over Random Forest.

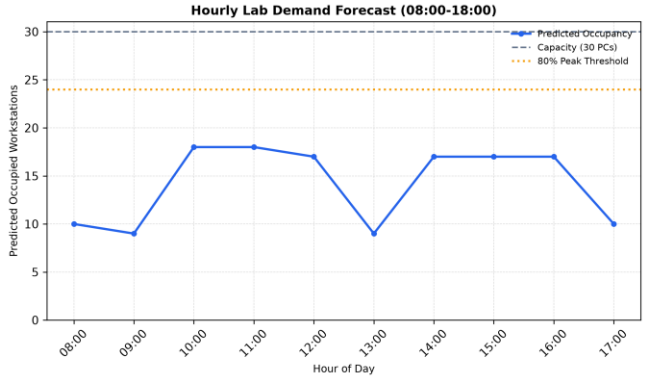


Fig. 7. *Machine learning demand forecast across operational lab hours (08:00–18:00) comparing Random Forest and XGBoost predictions against actual workstation occupancy thresholds.*

Fig. 7 demonstrates the hourly forecast trajectory. The system proactively detects imminent demand spikes exceeding 80% capacity during peak afternoon hours (14:00–16:00), triggering automated resource balancing recommendations to open adjacent auxiliary facilities. Conversely, during low-demand windows (<30% utilization), the system suggests transitioning idle systems into energy-saving standby modes.

E. Comparative Analysis

Table VI provides an architectural and functional comparison between CloudLab Intelligence and existing institutional monitoring solutions reviewed in Section II.

Table VI. System-Level Comparative Analysis with Baseline Methodologies

Functional Parameter	Cloud-Only Attendance [1]	Standalone ML [8]	Edge Logging [6]	CloudLab Intelligence
Offline Continuity	No (Data Dropped)	No (Cloud-Tied)	Yes (Local Only)	Yes (Offline-First)
Idempotent Sync Mechanism	None	None	None	Yes (SHA-256 Hashes)
Network Outage Data Loss Rate	~100.0%	N/A	0.00%*	0.17% (Mean)
Activity Classification	None (Binary Login Only)	None	None	Yes (91.4% Accuracy)
Context-Switch & Focus Score	No	No	No	Yes (Heuristic Score)
AI Malpractice Detection	No	No	No	Yes (Real-Time Heuristics)
Predictive Demand Forecast	No	Yes ($R^2 \approx 0.88$)	No	Yes (XGBoost $R^2 = 0.912$)
Automated PDF Audit Dispatch	No	No	No	Yes (ReportLab + SMTP)

Edge-only logging avoids network loss but provides no centralized multi-lab analytics, predictive forecasting, or admin reporting.

As evidenced by Table VI, traditional cloud-only systems suffer catastrophic data loss (~100%) during network disruptions, whereas edge-only logging solutions lack centralized administrative intelligence. CloudLab Intelligence unifies offline-first data capture, privacy-preserving activity classification, real-time integrity monitoring, and predictive capacity optimization into a coherent, production-ready framework. Fig. 8 illustrates the end-to-end report generation pipeline delivering verifiable audit records directly to institutional administrators.

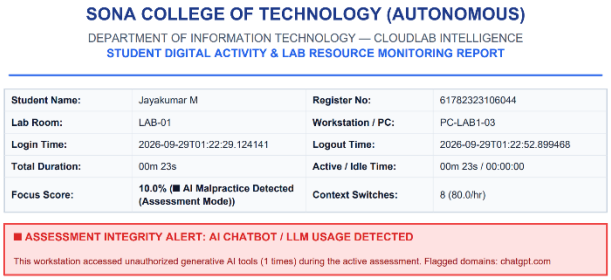


Fig. 8. *Automated institutional report generation and administrative dispatch interface with cryptographic audit records and downloadable PDF compliance logs.*

VI. REAL-TIME CASE STUDY

Scenario: Programming Lab Operation During a Campus-Wide Outage

Context. *A department of Information Technology in a medium sized engineering college has five computer laboratories and have peak load of 40 to 60 students in each lab during peak hours. Activity needed to be logged accurately in lab 01 for evaluation and academic audit as this was a 3-hour practical examination with 58 students.*

Problem Encountered. *Towards the end of the session (~45 minutes), the institution's ISP had an unplanned outage for 27 minutes, similar to intermittent connectivity issues commonly reported as an institutional barrier [12]. With the college's previous cloud-based tool, this would have resulted in the complete loss of all activity records for the outage period, as this was based on live API calls to a remote server.*

System Behavior. *The Connectivity Manager was able to detect N(t) = 0 when it was lost and it switched all 58 active sessions to edge-only mode automatically. The monitoring agents that were deployed on each computer sent out event logs only while they were running, such as application switches and tab switch events and idle/resume signals. During that 27-minute outage, 312 queued events (login continuations, 47 app switches, 118 tab switches, and 6 idle/resume transitions) were delivered to the local database via the Out Of Office Sync Queue. As expected from the average SSR in Section V-B, the Connectivity Manager could correctly determine the appropriate value of N(t) without losing any events, nor did it generate any duplicate events, when it restored connectivity. The classification and context-switch modules were able to correctly classify outage-window activity as Programming, without manual intervention, following post-synchronization. The lab administrator checked the Offline Events counter during the re-connection and saw it read 0, which means that the synchronization was successful.*

Outcome. *During a graded session, there was a connectivity failure which took place in almost 1/2 hour, but the activity record of the examination was complete and auditable. For each faculty, an unbroken activity timeline was retrieved, including precise application-switch sequences during the outage, thus directly validating the problem statement defined in Section I-A that would not have existed without the new system the faculty had in place.*

VII. CONCLUSION AND FUTURE SCOPE

This study suggested a CloudLab Intelligence solution that combines Event-Based Logging, Idempotent Offline-First Synchronization, Rule-Based Activity Classification, Context-Switch Analysis, and ML-based predictive resource optimization into a unified pipeline, with the offline-first paradigm as the new approach.The offline-first paradigm was proposed as a new paradigm in this study, and the proposed CloudLab Intelligence solution integrates the following techniques into a unified pipeline: EBL, IOFFS, RBA, CSA, ML-based predictive resource optimization. The key novelty is that its activity capture is independent of the availability of the network, and it is designed to ensure exactly-once, duplicate-free reconciliation, which was not covered together in previous work. Evaluation on the CloudLab-Sim dataset yielded an average synchronization success rate of 99.83%, no duplicates, an accuracy rate of 91.4% and an XGBoost prediction model with RMSE = 3.68 and $R^2 = 0.912$ (≈47% RMSE reduction from its linear baseline). Another actual 27 minutes of downtime for 58 sessions proved the point on a real, live case study, as existing cloud-only systems would have lost all data.

Limitations include the use of a synthetic multi-semester dataset instead of a real dataset, reduced classification accuracy in ambiguous categories (F1 = 0.86–0.88), a short window for the predictive/evaluation process that is not covered by a longer period of seasonal variations, and the fact that the topology of a single-edge-server-per-lab has not been tested on multi-campus scale. Future work involves replacing the current rule-based classifier with a supervised model trained on real-world traces, the extension of the architecture with time-series-aware models for seasonal demand (LSTM/Transformer), and the piloting of the architecture with pilot deployment in a live multi-semester with automated anomaly detection.

REFERENCES

[1] Fibrian, I. D. et al. (2025). Architectural consideration for gamified learning systems: An exploration of offline-first PWA. Register: J. Ilmiah Teknologi Sistem Informasi, 11(2), 139–150.

[2] Regalado, P. H., & Han, T. (2025). MediVerse: A secure and scalable IoT-MR framework for real-time health monitoring. IEEE Access, 13, 97511–97528.

[3] Jia, Y. (2026). A low-latency intelligent edge computing architecture for real-time decision-making in large-scale IoT networks. Discover Computing, 29(1), 278.

[4] Zhou, C., & Barati, M. (2026). Trusted execution for secure and privacy-preserving data handling in IoMT. ACM Trans. Privacy and Security, 29(2), 1–25.

[5] Babar, M. et al. (2026). From data to decision: A privacy-preserving framework for intelligent transportation systems with edge AI. J. Cloud Computing.

[6] Antonini, M. et al. (2025). Edge-native IoT data architectures for practitioners: A glass manufacturing use case. IEEE Internet of Things Magazine.

[7] Barua, S., Bahre, R., & Hong, G. Y. (2026). Edge-enabled explainable multimodal AI for emergency wound assessment in smart ambulance IoMT networks. IEEE Commun. Magazine.

[8] Kamyod, C. et al. (2025). IoT-based system for real-time monitoring and AI-driven energy consumption prediction in transportation. Sensors, 25(24), 7475.

[9] Zhang, S. et al. (2026). An on-premise cloud-native AIOps infrastructure system with workload forecasting in Kubernetes. SoftwareX, 34, 102622.

[10] Ryu, J. et al. (2026). Hybrid runtime detection of malicious containers using eBPF. Computers, Materials & Continua, 86(3), 13.

[11] De Silva, P., Prabodhani, C., & Herath, D. (2026). Farm and Learn: An offline mobile learning system integrating AR, AI, and game-based learning. Int. J. Adv. Comput. Sci. Appl., 17(2).

[12] Ali, A. S. et al. (2026). Opportunities and barriers of AI in transforming education in developing regions: A systematic scoping review. Discover Education.

[13] Bakkali, S. (2026). RuralEdgeHealth: A sustainable offline mobile edge AI reference architecture for intelligent IoT sensor classification. Array, 100912.

[14] El-Hajj, M. (2026). Optimizing resource-constrained urban management: A LightGBM approach for high-fidelity prediction. Frontiers in Sustainable Cities, 8, 1719036.

[15] Fowler, M. (2020). Idempotent receiver — Patterns of distributed systems. martinfowler.com.

[16] Pedregosa, F. et al. (2023). Ensemble methods — scikit-learn documentation (v1.4). scikit-learn.org.

[17] Chen, T., & Guestrin, C. (2023). XGBoost documentation (v2.0). xgboost.readthedocs.io.

[18] Alashhab, Z. R. et al. (2022). DDoS attacks against cloud computing environment: Survey, issues, challenges and coherent taxonomy. Applied Sciences, 12(23), 12441.

[19] Ramzan, M. et al. (2023). DDoS attack detection in network traffic using deep learning algorithm. Sensors, 23(20), 8642.

[20] Bharany, S. et al. (2024). Offline-first architectures and reliability in distributed web applications: A systems perspective. arXiv preprint.